Министерство образования Республики Беларусь
УО “Брестский государственный технический университет”
Кафедра ИИТ

Лабораторная работа №2
По дисциплине: «СПП»
Вариант 9

Выполнила:
студент 3 курса
группы ПО-12
Кравченко М.А.
Проверил:
Кулик А.Д.

Брест 2026

Цель работы: закрепить навыки объектно-ориентированного программирования на языке Python.

Задание 1.

Реализовать простой класс: множество вещественных чисел переменной мощности – Предусмотреть возможность объединения двух множеств, вывода на консоль элементов множества, а также метод, определяющий, принадлежит ли указанное значение множеству. Класс должен содержать методы, позволяющие добавлять и удалять элемент в/из множества. Конструктор должен позволять создавать объекты с начальной инициализацией. Реализацию множества осуществить на базе списка. Переопределить метод __eq__, выполняющий сравнение объектов данного типа.

Требования к выполнению

• Реализовать пользовательский класс по варианту.

Для каждого класса

• Создать атрибуты (поля) классов

• Создать методы классов

• Добавить необходимые свойства и сеттеры (по необходимости)

• Переопределить магические методы __str__ и __eq__

Код программы:

```
"""Модуль для работы с множеством вещественных чисел."""


class FloatSet:
    """Класс для представления множества вещественных чисел."""

    def __init__(self, *args):
        """Конструктор класса."""
        self._elements = []
        for num in args:
            self.add(num)

    @property
    def elements(self):
        """Геттер для получения элементов."""
        return self._elements.copy()

    @property
    def size(self):
        """Размер множества."""
        return len(self._elements)

    def add(self, value):
        """Добавляет элемент в множество."""
        try:
            value = float(value)
            if value not in self._elements:
                self._elements.append(value)
                return True
            return False
        except (TypeError, ValueError) as exc:
            raise TypeError("Множество может содержать только вещественные числа") from exc
```

```
    def remove(self, value):
        """Удаляет элемент из множества."""
        try:
            value = float(value)
            if value in self._elements:
                self._elements.remove(value)
                return True
            return False
        except (TypeError, ValueError) as exc:
            raise TypeError("Множество может содержать только вещественные числа") from exc

    def contains(self, value):
        """Проверяет принадлежность элемента."""
        try:
            return float(value) in self._elements
        except (TypeError, ValueError):
            return False

    def union(self, other):
        """Объединяет два множества."""
        result = FloatSet()
        for elem in self._elements:
            result.add(elem)
        for elem in other.elements:
            result.add(elem)
        return result

    def __str__(self):
        """Строковое представление."""
        if not self._elements:
            return "{}"
        elements_str = ", ".join(str(elem) for elem in sorted(self._elements))
        return "{" + elements_str + "}"

    def __eq__(self, other):
        """Сравнение множеств."""
        if not isinstance(other, FloatSet):
            return False
        return sorted(self._elements) == sorted(other._elements)

    def __contains__(self, value):
        """Оператор in."""
        return self.contains(value)

    def __len__(self):
        """Возвращает количество элементов в множестве."""
        return len(self._elements)


def print_menu():
    """Выводит меню."""
    print("\n" + "=" * 50)
    print("МЕНЮ УПРАВЛЕНИЯ МНОЖЕСТВОМ")
    print("=" * 50)
    print("1. Показать текущее множество")
    print("2. Добавить элемент")
    print("3. Удалить элемент")
    print("4. Проверить принадлежность элемента")
    print("5. Объединить с другим множеством")
    print("6. Сравнить с другим множеством")
    print("7. Создать новое множество")
    print("8. Выход")
    print("-" * 50)


def get_numbers_from_input(prompt):
    """Получает список чисел из ввода пользователя."""
    print(prompt)
```

```
    user_input = input("> ").strip()

    numbers = []
    if user_input:
        for item in user_input.split():
            try:
                numbers.append(float(item))
            except ValueError:
                print(f"  '{item}' не число, пропускаем")

    return numbers


def handle_show_set(my_set):
    """Обрабатывает показ множества."""
    print(f"\nМножество: {my_set}")
    elements = sorted(my_set.elements)
    print(f"Элементы: {', '.join(str(x) for x in elements)}")
    print(f"Количество элементов: {len(my_set)}")
    return my_set


def handle_add_element(my_set):
    """Обрабатывает добавление элемента."""
    try:
        value = float(input("Введите число для добавления: "))
        if my_set.add(value):
            print(f"Число {value} добавлено")
        else:
            print(f"Число {value} уже есть в множестве")
    except ValueError:
        print("Ошибка: введите число")
    except TypeError as e:
        print(f"{e}")
    return my_set


def handle_remove_element(my_set):
    """Обрабатывает удаление элемента."""
    try:
        value = float(input("Введите число для удаления: "))
        if my_set.remove(value):
            print(f"Число {value} удалено")
        else:
            print(f"Число {value} не найдено")
    except ValueError:
        print("Ошибка: введите число")
    return my_set


def handle_check_element(my_set):
    """Обрабатывает проверку принадлежности элемента."""
    try:
        value = float(input("Введите число для проверки: "))
        if value in my_set:
            print(f"Число {value} есть в множестве")
        else:
            print(f"Числа {value} нет в множестве")
    except ValueError:
        print("Ошибка: введите число")
    return my_set


def handle_union_sets(my_set):
    """Обрабатывает объединение множеств."""
    numbers = get_numbers_from_input("\nВведите числа для второго множества (через пробел):")
    other_set = FloatSet(*numbers)
```

```
    if len(other_set) > 0:
        print(f"Первое множество: {my_set}")
        print(f"Второе множество: {other_set}")

        union_set = my_set.union(other_set)
        print(f"Результат объединения: {union_set}")

        replace = input("Заменить текущее множество результатом? (д/н): ").lower()
        if replace in ["д", "да", "у", "yes"]:
            my_set = union_set
            print("Текущее множество обновлено")
    else:
        print("Второе множество пустое")
    return my_set


def handle_compare_sets(my_set):
    """Обрабатывает сравнение множеств."""
    numbers = get_numbers_from_input("\nВведите числа для второго множества (через пробел):")
    other_set = FloatSet(*numbers)

    print(f"Текущее множество: {my_set}")
    print(f"Второе множество: {other_set}")

    if my_set == other_set:
        print("Множества РАВНЫ")
    else:
        print("Множества НЕ РАВНЫ")
    return my_set


def handle_new_set(my_set):
    """Обрабатывает создание нового множества."""
    numbers = get_numbers_from_input("\nВведите числа для нового множества (через пробел):")
    my_set = FloatSet(*numbers)
    print(f"Создано новое множество: {my_set}")
    return my_set


def handle_exit():
    """Обрабатывает выход из программы."""
    print("\nПрограмма завершена. До свидания!")
    return True


def main():
    """Основная функция."""
    print("=" * 60)
    print("ПРОГРАММА ДЛЯ РАБОТЫ С МНОЖЕСТВОМ ВЕЩЕСТВЕННЫХ ЧИСЕЛ")
    print("=" * 60)

    # Создаем начальное множество
    numbers = get_numbers_from_input("\nВведите числа для начального множества (через пробел):")
    my_set = FloatSet(*numbers)
    print(f"Создано множество: {my_set}")

    exit_program = False
    while not exit_program:
        print_menu()
        print(f"Текущее множество: {my_set}")

        choice = input("\nВыберите действие (1-8): ").strip()

        if choice == "1":
            my_set = handle_show_set(my_set)
        elif choice == "2":
            my_set = handle_add_element(my_set)
        elif choice == "3":
```

```
            my_set = handle_remove_element(my_set)
        elif choice == "4":
            my_set = handle_check_element(my_set)
        elif choice == "5":
            my_set = handle_union_sets(my_set)
        elif choice == "6":
            my_set = handle_compare_sets(my_set)
        elif choice == "7":
            my_set = handle_new_set(my_set)
        elif choice == "8":
            exit_program = handle_exit()
        else:
            print("Неверный выбор. Введите число от 1 до 8.")

        if not exit_program:
            input("\nНажмите Enter, чтобы продолжить...")


if __name__ == "__main__":
    try:
        main()
    except KeyboardInterrupt:
        print("\n\nПрограмма прервана пользователем. До свидания!")
```

```
==================================================
МЕНЮ УПРАВЛЕНИЯ МНОЖЕСТВОМ
==================================================
1. Показать текущее множество
2. Добавить элемент
3. Удалить элемент
4. Проверить принадлежность элемента
5. Объединить с другим множеством
6. Сравнить с другим множеством
7. Создать новое множество
8. Выход
--------------------------------------------------
Текущее множество: {1.12, 10.6, 10.68, 15.55}

Выберите действие (1-8): 5

Введите числа для второго множества (через пробел):
> 3.35 8.86 17.777 12.2
Первое множество: {1.12, 10.6, 10.68, 15.55}
Второе множество: {3.35, 8.86, 12.2, 17.777}
Результат объединения: {1.12, 3.35, 8.86, 10.6, 10.68, 12.2, 15.55, 17.777}
Заменить текущее множество результатом? (д/н): д
✅ Текущее множество обновлено
```

Задание 2.

Построить модель программной системы с применением отношений (обобщения, агрегации, ассоциации, реализации) между классами. Задать атрибуты и методы классов. Реализовать (если необходимо) дополнительные классы. Продемонстрировать работу разработанной системы.

Система Железнодорожная касса. Пассажир делает Заявку на станцию назначения, время и дату поездки. Система регистрирует Заявку и осуществляет поиск подходящего Поезда. Пассажир делает выбор Поезда и получает Счет на Оплату. Администратор вводит номера Поездов, промежуточные и конечные станции, цены.

Код программы:

```
"""
Система Железнодорожная касса.
"""
```

```python
from abc import ABC, abstractmethod
from typing import List, Dict


class Payable(ABC):
    """Интерфейс для оплаты."""

    @abstractmethod
    def execute_payment(self) -> bool:
        """Выполнить оплату."""

    @abstractmethod
    def get_payment_status(self) -> bool:
        """Получить статус оплаты."""

    @abstractmethod
    def get_payment_amount(self) -> float:
        """Получить сумму оплаты."""


class PaymentSystem(Payable):
    """Платежная система."""

    def __init__(self, amount_value: float):
        self.amount_value = amount_value
        self.payment_status = False

    def execute_payment(self) -> bool:
        """Выполнить оплату."""
        print(f"  Оплата {self.amount_value} руб...")
        self.payment_status = True
        return True

    def get_payment_status(self) -> bool:
        """Получить статус оплаты."""
        return self.payment_status

    def get_payment_amount(self) -> float:
        """Получить сумму оплаты."""
        return self.amount_value


class User:
    """Базовый класс пользователя."""

    def __init__(self, user_identifier: int, user_fullname: str):
        self.identifier = user_identifier
        self.fullname = user_fullname

    def get_fullname(self) -> str:
        """Получить имя."""
        return self.fullname

    def get_identifier(self) -> int:
        """Получить ID."""
        return self.identifier


class Passenger(User):
    """Пассажир."""

    def __init__(self, user_identifier: int, user_fullname: str, passport_data: str):
        super().__init__(user_identifier, user_fullname)
        self.passport_data = passport_data
        self.purchased_tickets = []

    def create_request(self, destination: str, date: str, time: str):
        """Создать заявку."""
```

```
        return Request(self, destination, date, time)

    def add_ticket(self, ticket):
        """Добавить билет."""
        self.purchased_tickets.append(ticket)

    def show_tickets(self):
        """Показать билеты."""
        if not self.purchased_tickets:
            print("  Нет билетов")
            return
        print("\nБилеты:")
        for t in self.purchased_tickets:
            print(f"  {t}")

    def get_tickets_count(self) -> int:
        """Количество билетов."""
        return len(self.purchased_tickets)

    def get_passport(self) -> str:
        """Паспорт."""
        return self.passport_data


class Administrator(User):
    """Администратор."""

    def __init__(self, user_identifier: int, user_fullname: str, job_position: str):
        super().__init__(user_identifier, user_fullname)
        self.job_position = job_position

    def get_position(self) -> str:
        """Должность."""
        return self.job_position

    def change_position(self, new_position: str):
        """Изменить должность."""
        self.job_position = new_position


class Train:
    """Поезд."""

    def __init__(self, train_number: str, route: List[str], prices: Dict[str, float]):
        self.train_number = train_number
        self.route = route
        self.prices = prices
        self.reserved = {}

    def get_price(self, destination: str):
        """Цена до станции."""
        return self.prices.get(destination)

    def is_available(self, date: str):
        """Проверить наличие мест."""
        return date not in self.reserved

    def reserve(self, date: str):
        """Забронировать место."""
        self.reserved[date] = True

    def get_number(self) -> str:
        """Номер поезда."""
        return self.train_number

    def get_route(self) -> List[str]:
        """Маршрут."""
        return self.route.copy()
```

```
    def get_destinations(self) -> List[str]:
        """Список назначений."""
        return list(self.prices.keys())

    def __str__(self):
        return f"Поезд {self.train_number}: {' -> '.join(self.route)}"


class Request:
    """Заявка."""

    def __init__(self, passenger: Passenger, dest: str, date: str, time: str):
        self.passenger = passenger
        self.destination = dest
        self.date = date
        self.time = time
        self.found = []

    def search(self, office):
        """Поиск поездов."""
        print(f"\nПоиск до {self.destination}...")
        for train in office.get_all_trains():
            if train.get_price(self.destination) and train.is_available(self.date):
                self.found.append(train)
        return self.found.copy()

    def get_found(self):
        """Получить найденные."""
        return self.found.copy()

    def get_destination(self):
        """Станция."""
        return self.destination

    def get_passenger(self):
        """Пассажир."""
        return self.passenger

    def get_date(self):
        """Дата."""
        return self.date

    def get_time(self):
        """Время."""
        return self.time

    def has_results(self) -> bool:
        """Есть результаты."""
        return len(self.found) > 0


class Ticket:
    """Билет."""

    _counter = 1000

    def __init__(self, ticket_data: dict):
        """Создание билета из словаря."""
        self.owner = ticket_data["owner"]
        self.train = ticket_data["train"]
        self.dest = ticket_data["dest"]
        self.date = ticket_data["date"]
        self.price = ticket_data["price"]
        Ticket._counter += 1
        self.code = f"T{Ticket._counter}"

    def get_code(self) -> str:
```

```
        """Код билета."""
        return self.code

    def get_owner(self) -> str:
        """Владелец."""
        return self.owner

    def get_price(self) -> float:
        """Цена."""
        return self.price

    def __str__(self):
        return f"Билет {self.code}: {self.owner} -> {self.dest}, " f"{self.date}, {self.price} руб."


class Invoice:
    """Счет."""

    _counter = 1000

    def __init__(self, customer: str, dest: str, train: str, amount: float):
        self.customer = customer
        self.dest = dest
        self.train = train
        self.amount = amount
        Invoice._counter += 1
        self.code = f"I{Invoice._counter}"
        self.payment = None

    def create_payment(self):
        """Создать платеж."""
        self.payment = PaymentSystem(self.amount)
        return self.payment

    def get_amount(self):
        """Сумма."""
        return self.amount

    def get_customer(self):
        """Клиент."""
        return self.customer

    def get_train(self):
        """Поезд."""
        return self.train

    def get_dest(self):
        """Назначение."""
        return self.dest

    def get_code(self):
        """Номер счета."""
        return self.code

    def is_paid(self) -> bool:
        """Оплачен ли счет."""
        return self.payment is not None and self.payment.get_payment_status()


class TicketOffice:
    """Касса."""

    def __init__(self, name: str):
        self.name = name
        self.trains = []
        self.requests = []
        self.sold = []
        self.invoices = []
```

```
    def add_train(self, train):
        """Добавить поезд."""
        self.trains.append(train)

    def add_request(self, request):
        """Добавить заявку."""
        self.requests.append(request)

    def get_all_trains(self):
        """Все поезда."""
        return self.trains.copy()

    def show_trains(self):
        """Показать поезда."""
        print(f"\nПоезда '{self.name}':")
        if not self.trains:
            print("  Нет")
            return
        for i, t in enumerate(self.trains, 1):
            print(f"  {i}. {t}")

    def create_invoice(self, request, idx):
        """Создать счет."""
        found = request.get_found()
        if not found or idx < 1 or idx > len(found):
            return None
        train = found[idx - 1]
        price = train.get_price(request.get_destination())
        if price is None:
            return None
        inv = Invoice(request.get_passenger().get_fullname(),
                      request.get_destination(), train.get_number(), price)
        self.invoices.append(inv)
        return inv

    def process_payment(self, invoice):
        """Обработать оплату."""
        if invoice.is_paid():
            return None
        payment = invoice.create_payment()
        if payment.execute_payment():
            ticket_data = {
                "owner": invoice.get_customer(),
                "train": invoice.get_train(),
                "dest": invoice.get_dest(),
                "date": invoice.get_dest(),
                "price": invoice.get_amount(),
            }
            ticket = Ticket(ticket_data)
            self.sold.append(ticket)
            return ticket
        return None

    def get_trains_count(self) -> int:
        """Количество поездов."""
        return len(self.trains)

    def get_sold_count(self) -> int:
        """Количество проданных билетов."""
        return len(self.sold)

    def get_requests_count(self) -> int:
        """Количество заявок."""
        return len(self.requests)


def input_trains_data(office):
```

```
    """Ввод данных о поездах."""
    print("\nПОЕЗДА")
    cnt = int(input("Количество: "))

    for i in range(cnt):
        print(f"\n--- {i+1} ---")
        num = input("Номер: ")
        route = input("Маршрут (через пробел): ").split()
        prices = {}
        for s in route[1:]:
            prices[s] = float(input(f"Цена до {s}: "))
        office.add_train(Train(num, route, prices))


def handle_passenger_actions(office, passenger):
    """Обработка действий пассажира."""
    office.show_trains()

    print("\nЗАЯВКА")
    req = passenger.create_request(input("Куда: "), input("Дата (ГГГГ-ММ-ДД): "), input("Время: "))

    found = req.search(office)
    if not found:
        print("Ничего нет")
        return None

    print(f"\nНайдено: {len(found)}")
    for i, t in enumerate(found, 1):
        print(f"  {i}. {t.get_number()} - {t.get_price(req.get_destination())} руб.")

    choice = int(input(f"\nВыбрать (1-{len(found)}): "))
    inv = office.create_invoice(req, choice)
    if not inv:
        print("Ошибка")
        return None

    print(f"Сумма: {inv.get_amount()} руб.")
    return inv


def process_payment_flow(office, invoice, passenger):
    """Обработка оплаты."""
    if input("Платить? (д/н): ").lower() in ["д", "да"]:
        ticket = office.process_payment(invoice)
        if ticket:
            print("\nГОТОВО")
            print(ticket)
            passenger.add_ticket(ticket)
            return ticket
    return None


def show_final_stats(office, passenger):
    """Показать итоговую статистику."""
    trains_count = office.get_trains_count()
    sold_count = office.get_sold_count()

    print("\n" + "=" * 60)
    print(f"Поездов: {trains_count}")
    print(f"Продано: {sold_count}")
    passenger.show_tickets()
    print("=" * 60)


def main():
    """Главная функция."""
    print("=" * 60)
    print("ЖЕЛЕЗНОДОРОЖНАЯ КАССА")
```

```
    print("=" * 60)

    name = input("\nНазвание кассы: ")
    office = TicketOffice(name)

    print("\nАДМИНИСТРАТОР")
    admin_name = input("Имя: ")
    admin_pos = input("Должность: ")
    admin = Administrator(1, admin_name, admin_pos)
    print(f"Администратор {admin.get_fullname()} добавлен")

    input_trains_data(office)

    print("\nПАССАЖИР")
    p = Passenger(2, input("Имя: "), input("Паспорт: "))

    invoice = handle_passenger_actions(office, p)
    if invoice:
        process_payment_flow(office, invoice, p)

    show_final_stats(office, p)


if __name__ == "__main__":
    main()
```

```
【1】 СОЗДАНИЕ КАССЫ
Введите название кассы: Вайбвокзал
✅ Касса 'Вайбвокзал' создана

【2】 СОЗДАНИЕ АДМИНИСТРАТОРА
Имя админа: Марго
Логин: margo512
Должность: Админ
✅ Админ Марго создан

【3】 ДОБАВЛЕНИЕ ПОЕЗДОВ
Сколько поездов добавить? 2

--- Поезд 1 ---
Номер поезда: 1
Маршрут (станции через пробел):
> Брест Береза Минск
Цены до станций:
  Цена до Береза: 4
  Цена до Минск: 15
✅ Поезд 1 добавлен

--- Поезд 2 ---
Номер поезда: Брест Береза Белоозерск
Маршрут (станции через пробел):
> Брест Береза Белоозерск
Цены до станций:
  Цена до Береза: 4
  Цена до Белоозерск: 2
✅ Поезд Брест Береза Белоозерск добавлен

【4】 СОЗДАНИЕ ПАССАЖИРА
Имя пассажира: Оксимирон
Логин: okskrutaya
Паспорт: AB1234567
✅ Пассажир Оксимирон создан
```

```
【5】 ПРОСМОТР ПОЕЗДОВ

Поезда в системе 'Вайбвокзал':
  1. Поезд №1: Брест -> Береза -> Минск
  2. Поезд №Брест Береза Белоозерск: Брест -> Береза -> Белоозерск

【6】 СОЗДАНИЕ ЗАЯВКИ
Станция назначения: Береза
Дата (ГГГГ-ММ-ДД): 2026-03-25
Время: 13:00
✅ Заявка создана

【7】 ПОИСК ПОЕЗДОВ

🔍 Поиск поездов до Береза...

Найдено поездов: 2
  1. 1 - 4.0 руб.
  2. Брест Береза Белоозерск - 4.0 руб.

【8】 ВЫБОР ПОЕЗДА
Выберите поезд (1-2): 2
✅ Счет создан на сумму 4.0 руб.

【9】 ОПЛАТА
Оплатить? (да/нет): да
  💳 Оплата 4.0 руб...

✅ ПОКУПКА УСПЕШНА!
🎫 Билет №T3686: Оксимирон -> Береза, 2026-03-25, 4.0 руб.

============================================================
ИТОГИ:
Поездов в системе: 2
Билетов продано: 1

Ваши билеты:
  Билет №T3686: Оксимирон -> Береза, 2026-03-25, 4.0 руб.
============================================================
```

Вывод: закрепила навыки объектно-ориентированного программирования на языке Python.